Comparisons for `reference_wrapper`

Document #:	P2944R0
Date:	2023-07-08
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

Typically in libraries, wrapper types are comparable when their underlying types are comparable. `tuple<T>` is equality comparable when `T` is. `optional<T>` is equality comparable when `T` is. `variant<T>` is equality comparable when `T` is.

But `reference_wrapper<T>` is a peculiar type in this respect. It looks like this:

```
template<class T> class reference_wrapper {
public:
  // types
  using type = T;

  // [refwrap.const], constructors
  template<class U>
    constexpr reference_wrapper(U&&) noexcept(see below);
  constexpr reference_wrapper(const reference_wrapper& x) noexcept;

  // [refwrap.assign], assignment
  constexpr reference_wrapper& operator=(const reference_wrapper& x)
        noexcept;

  // [refwrap.access], access
  constexpr operator T&() const noexcept;
  constexpr T& get() const noexcept;

  // [refwrap.invoke], invocation
  template<class... ArgTypes>
    constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...)
        const
      noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);
};
```

When `T` is not equality comparable, it is not surprising that `reference_wrapper<T>` is not equality comparable. But what about when `T` *is* equality comparable? There are no comparison operators here, but nevertheless the answer is… maybe?

Because `reference_wrapper<T>` is implicitly convertible to `T&` and `T` is an associated type of `reference_wrapper<T>`, `T`'s equality operator (if it exists) might be viable candidate. But it depends on exactly what `T` is and how the equality operator is defined. Given a type `T` and an object `t` such that `t == t` is valid, let's consider the validity of the expressions `ref(t) == ref(t)` and `ref(t) == t` for various possible types `T`:

`T`	`ref(t) == ref(t)`	`ref(t) == t`
builtins	✔️	✔️
class or class template with member `==`	❌	✔️ (since C++20)
class with non-member or hidden friend `==`	✔️	✔️
class template with hidden friend `==`	✔️	✔️
class template with non-member, template `==`	❌	❌
`std::string_view`	❌	✔️

That's a weird table!

Basically, if `T` is equality comparable, then `std::reference_wrapper<T>` is… sometimes… depending on how `T`'s comparisons are defined. `std::reference_wrapper<int>` is equality comparable, but `std::reference_wrapper<std::string>` is not. Nor is `std::reference_wrapper<std::string_view>` but you can nevertheless compare a `std::reference_wrapper<std::string_view>` to a `std::string_view`.

So, first and foremost: sense, this table makes none.

Second, there are specific use-cases to want `std::reference_wrapper<T>` to be normally equality comparable, and those use-cases are the same reason what `std::reference_wrapper<T>` exists to begin with: deciding when to capture a value by copy or by reference.

Consider wanting to have a convenient shorthand for a predicate to check for equality against a value. This is something that shows up in lots of libraries (e.g. Björn Fahller's lift or Conor Hoekstra's blackbird), and looks something like this:

```
inline constexpr auto equals = [](auto&& value){
  return [value=FWD(value)](auto&& e){ return value == e; };
};
```

Which allows the nice-looking:

```
if (std::ranges::any_of(v, equals(0))) {
    // ...
}
```

But this implementation always copies (or moves) the value into the lambda. For larger types, this is wasteful. But we don't want to either unconditionally capture by reference (which sometimes leads to

dangling) or write a parallel hierarchy of reference-capturing function objects (which is lots of code duplication and makes the library just worse).

This is *exactly* the problem that `std::reference_wrapper<T>` solves for the standard library: if I want to capture something by reference into `std::bind` or `std::thread` or anything else, I pass the value as `std::ref(v)`. Otherwise, I pass `v`. We should be able to use the exact same solution here, without having to change the definition of `equals`:

```
if (std::ranges::any_of(v, equals(std::ref(target)))) {
    // ...
}
```

And this works! Just… only for some types, seemingly randomly. The goal of this proposal is for it to just always work.

§ 2 Proposal

Add `==` and `<=>` to `std::reference_wrapper<T>` so that `std::reference_wrapper<T>` is always comparable when `T` is, regardless of how `T`'s comparisons are defined.

Change 22.10.6.1 [refwrap.general]:

```
  template<class T> class reference_wrapper {
  public:
    // types
    using type = T;

    // [refwrap.const], constructors
    template<class U>
      constexpr reference_wrapper(U&&) noexcept(see below);
    constexpr reference_wrapper(const reference_wrapper& x) noexcept;

    // [refwrap.assign], assignment
    constexpr reference_wrapper& operator=(const reference_wrapper& x)
        noexcept;

    // [refwrap.access], access
    constexpr operator T& () const noexcept;
    constexpr T& get() const noexcept;

    // [refwrap.invoke], invocation
    template<class... ArgTypes>
      constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...)
        const
       noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);

+   // [refwrap.comparisons], comparisons
+   friend constexpr bool operator==(reference_wrapper, reference_wrapper);
+   friend constexpr synth-three-way-result<T> operator<=>
        (reference_wrapper, reference_wrapper);
  };
```

Add a new clause, [refwrap.comparisons], after 22.10.6.5 [refwrap.invoke]:

```
friend constexpr bool operator==(reference_wrapper x, reference_wrapper y);
```

1 *Constraints*: The expression `x.get() == y.get()` is well-formed and its result is convertible to `bool`.

2 *Returns*: `x.get() == y.get()`.

```
friend constexpr synth-three-way-result<T> operator<=>(reference_wrapper x,
        reference_wrapper y);
```

3 *Returns*: `synth-three-way(x.get()) <=> synth-three-way(y.get())`.

§ 2.1 Feature-test macro

We don't have a feature-test macro for `std::reference_wrapper<T>`, and there doesn't seem like a good one to bump for this, so let's add a new one to 17.3.2 [version.syn]

```
+ #define __cpp_lib_reference_wrapper 20XXXXL // also in <functional>
```